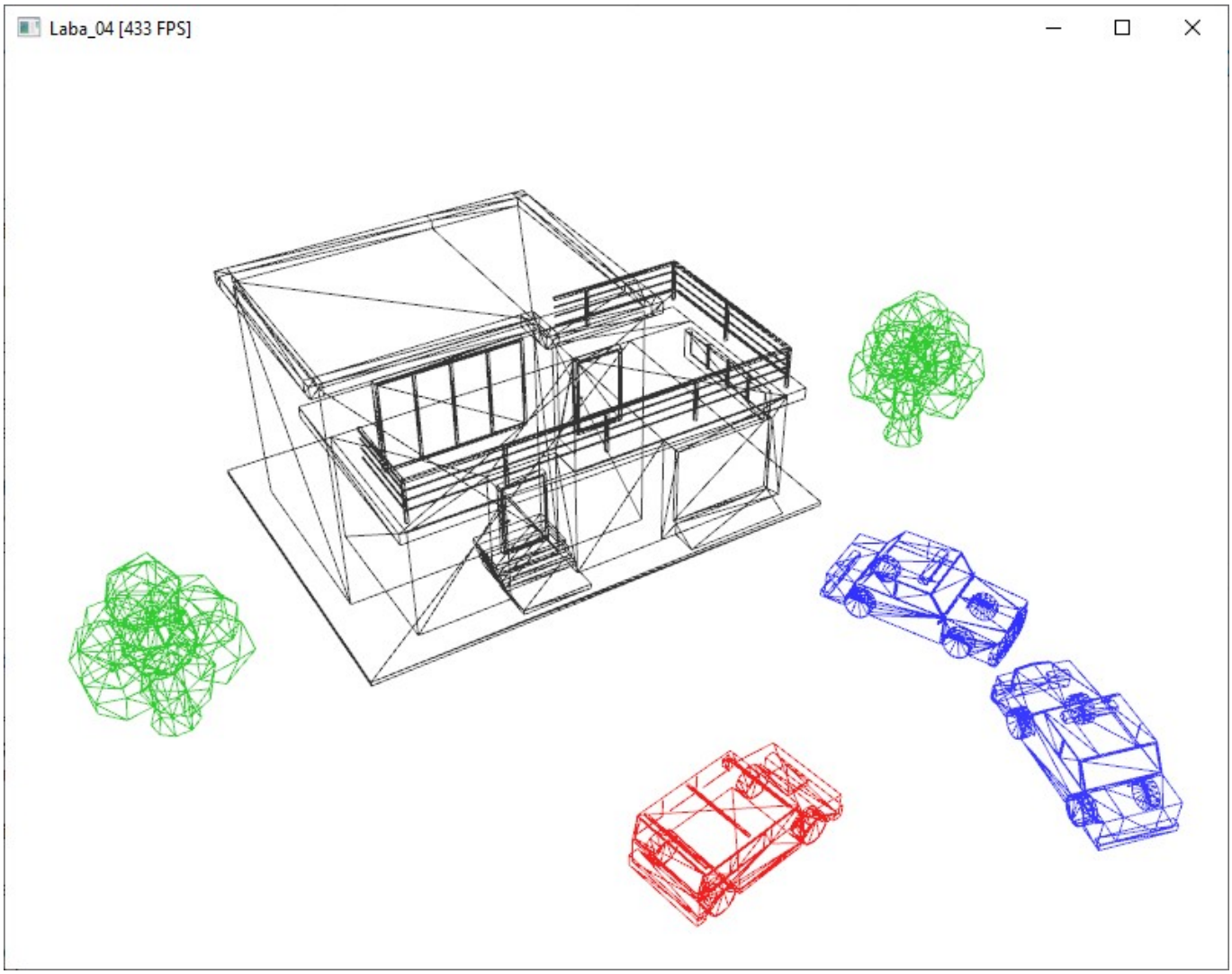
# Лабораторная работа №04

## Вывод полигонов с использованием VBO и VAO.

Целью лабораторной работы №4 является вывод на экран нескольких графических объектов, каждый из которых представлен некоторым мешем, загруженным из внешнего obj-файла. Для вывода меша на экран необходимо использованием буферы OpenGL и VAO-объекты. Ресурсы для демонстрации программы (меши объектов и машин) доступны в примере к лабораторной работе.



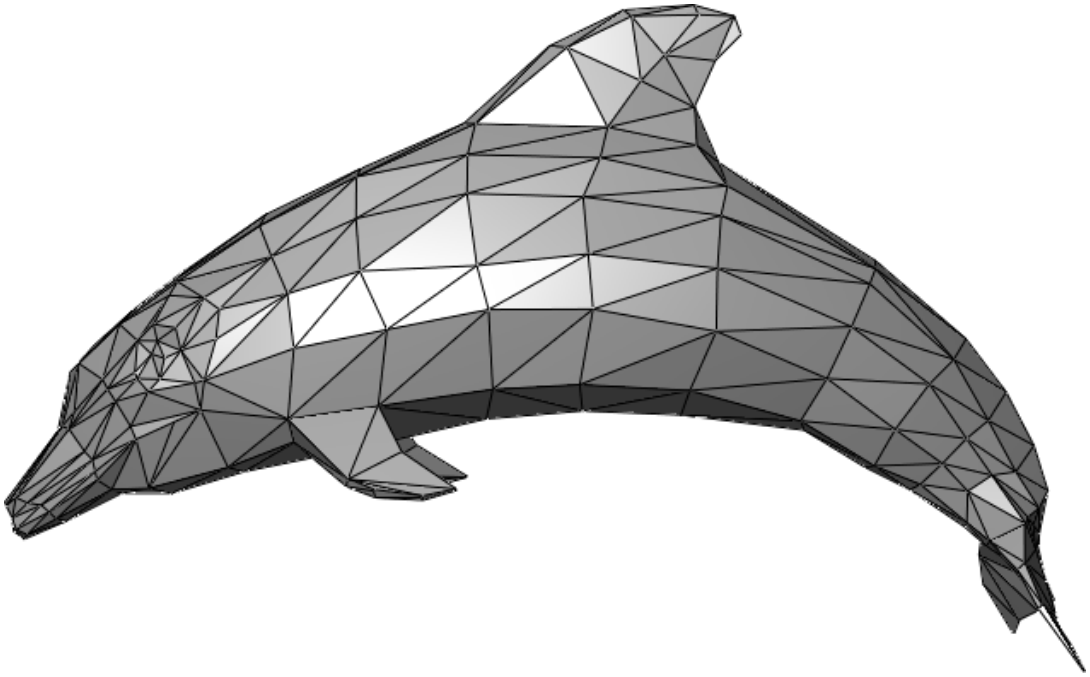

В процессе реализации программы, прежде всего, необходимо создать класс Mesh, отвечающий за загрузку меша из obj-файла, а также за его вывод на экран.

Кроме того, требуется реализовать класс ResourceManager, который необходим для централизованного управления всеми ресурсами (в частности, для хранения всех ресурсов в единственном экземпляре). На данном этапе к ресурсам относятся только меши, но в дальнейшем сюда же будут относиться материалы, текстуры и, возможно, другие данные. Класс для управления ресурсами организуется на основе шаблона проектирования singleton (одиночка). Освоение данного шаблона и реализация класса является наиболее значимой целью лабораторной работы.

В завершении необходимо внести изменения в класс GraphicObject, отвечающий за представление одного графического объекта. В данной лабораторной работе к графическому объекту будет добавлена информация об используемом меше (его идентификатор в менеджере ресурсов).

# Вывод полигонов в OpenGL.

Трехмерные графические объекты, выводимые на экран, представлены, как правило, группой полигонов, называемых полигональной сеткой или мешем (polygon mesh). Каждый отдельно взятый полигон является простейшей геометрической фигурой, которая может быть задана всеми своими вершинами. В качестве полигонов чаще всего используются треугольники, поскольку их создание, модификация, хранение и обработка является достаточно простым процессом.



Для задания каждого полигона, необходимо определить все вершины, составляющие данный полигон. Вершины, в свою очередь, описываются всеми своими атрибутами, среди которых могут быть:

- геометрические координаты вершины, описывающие её положение в пространстве;
- вектор нормали, который используется для расчета освещения объекта;
- текстурные координаты, необходимые для наложения текстур;
- множество других атрибутов, которые могут быть необходимы в каждом конкретном случае.

Таким образом, для представления любого трехмерного объекта с заданной точностью необходимо определить все вершины каждого полигона, а для этого необходимо задать все атрибуты каждой вершины. То есть, необходимо иметь в памяти массив вершин, который хранит все атрибуты каждой вершины. Вершина должна, как минимум, иметь геометрические координаты, задающие позицию вершины в некоторой системе координат. Для задания каждого отдельного меша, как правило, используется собственная локальная система координат (система координат модели).

При построении достаточно сложного объекта некоторые вершины могут принадлежать сразу нескольким полигонам. Поэтому, для экономии памяти, каждая вершина хранится в памяти в единственном экземпляре, а для указания того, из каких вершин строится каждый полигон, используется массив индексов.

Механизм использования массива вершин и массива индексов был подробно рассмотрен в предыдущем семестре, основные моменты использования данных массивов остались неизменными и в современной версии OpenGL – версии, которая задействует механизм шейдеров для преобразования вершин и расчета цвета фрагментов.

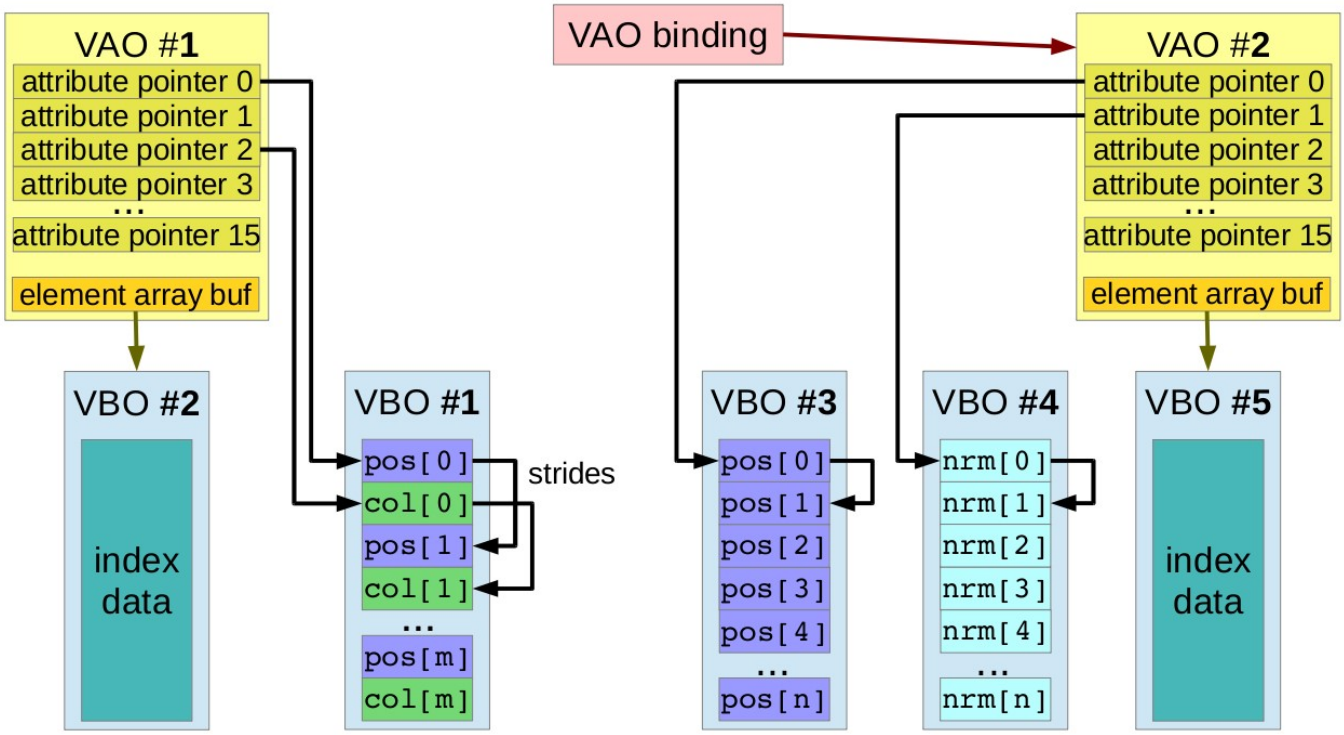
Тем не менее, в современной версии OpenGL в механизм хранения и обработки вершин были добавлены некоторые изменения, часть из которых связанна с тем, что с введением шейдеров было принято решение отказаться от предопределенных атрибутов вершин и позволить программистам самостоятельно определять нужные атрибуты. Кроме того, были добавлены еще некоторые возможности, ускоряющие процесс вывода полигонов. В частности, для вывода модели на экран в современной версии OpenGL используются две основные сущности (две внутренние структуры данных):

1. **Буферные объекты OpenGL** – используются для хранения различных данных в памяти видеокарты, таких как, например, массив атрибутов вершин или массив индексов. Буферы хранят данные в памяти видеокарты и при необходимости могут быть использованы OpenGL для получения этих данных. Буферы OpenGL бывают нескольких типов, для вывода полигонов применяются буфер вершин (GL_ARRAY_BUFFER) и буфер индексов (GL_ELEMENT_ARRAY_BUFFER).

   Буфер первого типа позволяет хранить атрибуты вершин, такие как: геометрические координаты, текстурные координаты, вектора нормалей и так далее. Данный тип буфера используется командой glVertexAttribPointer, то есть атрибуты вершины извлекаются из ранее указанного буфера вершин (буфера типа GL_ARRAY_BUFFER).

   Буфер второго типа позволяет хранить индексы. Данный тип буфера используется командой glDrawElements, то есть при выводе полигонов с использованием этой команды индексы извлекаются из указанного буфера индексов (GL_ELEMENT_ARRAY_BUFFER).

2. **VAO-объект (Vertex Array Object)** – используется для указания того, из какого буфера VBO берется каждый атрибут вершины, как он хранится в этом буфере, а также для указания того, в какую переменную шейдера передается соответствующий атрибут. Единожды инициализированный VAO-объект сохраняет свое состояние, то есть запоминает то, откуда необходимо брать данные для построения полигонов, и может быть использован повторно в любое время для вывода модели.




Для лучшего понимания назначения каждой из этих сущностей, можно привести аналогию со строительством дома. Если представить, что вывод трехмерного объекта на экран – это строительство дома, то буферы VBO – это склад материалов. Перед началом строительства дома необходимо заполнить эти склады всеми необходимыми материалам, то есть необходимо поместить в память видеокарты массив вершин и массив индексов. До того, как дом будет построен (объект будет выведен на экран), все эти материалы (данные) просто хранятся на складе (в памяти видеокарты).

Непосредственно для построения дома необходим план - инструкция, которая объясняет, откуда следует брать материалы, в каком порядке это следует делать и как их необходимо комбинировать между собой, для получения нужного результата. В качестве такого плана для вывода трехмерного объекта на экран выступает VAO-объект. VAO-объект показывает, из какого буфера следует извлекать данные, сколько данных следует извлекать и куда их необходимо передавать. Используя один, предварительно составленный план, можно вывести столько объектов, сколько требуется и столько раз, сколько необходимо. Совершенно очевидно, что план (VAO-объект) занимает гораздо меньше места, чем склад с материалами (VBO-буфер).

Таким образом, для вывода трехмерных объектов необходимы буферы VBO, которые содержат данные, и VAO-объекты, которые описывают, как эти данные следует использовать. При этом данные, необходимые для представления меша, могут генерироваться программно, что было продемонстрированно в первых двух лабораторных работах, где в качестве мешей выступал прямоугольник и куб. Однако на практике меши, в большинстве случаев, загружаются из внешних файлов одного из графических форматов. В частности, в данной лабораторной работе необходимо организовать вывод на экран мешей, загруженных из файла формата obj. Структура файла и способ его загрузки были рассмотрены в проекте прошлого семестра.

В результате, работу с мешем можно разделить на три этапа:

1. Загрузка меша из внешнего файла в оперативную память – данный этап выполняется, как правило, один раз либо при инициализации программы, либо при первом обращении к соответствующему мешу.
2. Инициализация VBO-буферов и VAO-объекта – данный этап выполняется совместно с первым этапом и заключается в создании всех необходимых буферов VBO и VAO-объекта для работы с мешем. Инициализированные на этом этапе данные будут использоваться на следующем этапе для вывода меша на экран в каждом кадре.
3. Вывод меша - выбор ранее инициализированного VAO-объекта и вызов одной из функций для вывода полигонов на экран (glDrawArrays, glDrawElements и так далее). При этом атрибуты вершины извлекаются в соответствии с VAO-объектом из ранее указанных буферов, ранее указанным способом.

## Загрузка меша из внешнего файла в оперативную память.

Первым этапом является загрузка меша из obj-файла. Структура файла, рекомендации по его загрузке и преобразованию были подробно рассмотрены в одной из лабораторных работ прошлого семестра. Для представления одной вершины со всеми её атрибутами рекомендуется использовать структуру следующего формата:

```
// Структура, описывающая одну вершину полигональной сетки
// каждая вершина имеет геометрические координаты,
// вектор нормали и текстурные координаты
struct Vertex
{
	// геометрические координаты
	GLfloat coord[3];
	// вектор нормали
	GLfloat normal[3];
	// текстурные координаты нулевого текстурного блока
	GLfloat texCoord[2];

};
```

В результате считывания и преобразования исходного файла в оперативной памяти должен быть сформирован массив верши и массив индексов. Поскольку эти данные необходимы только до тех пор, пока они не будут переданы в буферы VBO, нет необходимости делать их полями класса. Более того, после их передачи в OpenGL указанные данные должны быть удалены, чтобы не занимать место в оперативной памяти.

## Инициализация VBO-буферов и VAO-объекта.

Второй этап выполняется, как правило, вместе с первым. На данном этапе происходит создание и инициализация VAO-объекта, который впоследствии будет использоваться для вывода модели на экран, а также создание и заполнение буфера вершин и буфера индексов данными, полученными на предыдущем этапе. Для выполнения указанных действий необходимо использовать следующую последовательность действий:

***1. Создание и привязка VAO-объекта.***

Прежде всего, необходимо создать новый VAO-объект с помощью команды glGenVertexArrays. Данная функция создает нееобходимое количество VAO-объектов и записывает их идентификаторы в массив, адрес которого передан в качестве параметра. Данные идентификаторы необходимо запомнить, чтобы впоследствии указывать OpenGL, какой именно VAO-объект используется для выполнения определенных действий, например, для вывода объектов на экран. Описание функции и её параметров можно найти по следующей ссылке:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glGenVertexArrays.xhtml

Далее необходимо сделать вновь созданный VAO-объект текущим для того, чтобы все последующие команды модифицировали именно этот объект. Для выполнения указанного действия необходимо воспользоваться функцией glBindVertexArray:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glBindVertexArray.xhtml

***2. Создание и инициализация буфера вершин.***

Затем необходимо создать и инициализировать буфер вершин. Для этого последовательно применяются функции glGenBuffers, glBindBuffer и glBufferData. Данный процесс мало чем отличается от инициализации буфера вершин, используемой в проекте предыдущего семестра. Тем не менее, следует отметить, что теперь информация о привязке буферов OpenGL сохраняется в ранее выбранном VAO-объекте, то есть VAO-объект показывает, какой буфер используется для построения треугольников. Именно поэтому «отвязывать» буферы VBO после завершения инициализации не следует, иначе эта информация будет потерянна. Более подробную информацию по этим функциям можно получить по следующим ссылкам с официального сайта OpenGL:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glGenBuffers.xhtml

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glBindBuffer.xhtml

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glBufferData.xhtml

***3. Создание буфера индексов.***

Следующим этапом является создание буфера индексов. Создание буфера индексов выполняется аналогично созданию буфера вершин, однако используется другой тип буфера - GL_ELEMENT_ARRAY_BUFFER вместо GL_ARRAY_BUFFER.

***4. Указание места и формата хранения атрибутов вершин.***

Далее необходимо указать, откуда и как извлекать каждый атрибут вершины при построении полигонов, а также в какую переменную вершинного шейдера передавать указанные данные. Ранее для этого использовались команды glVertexPointer, glColorPointer, glNormalPointer и так далее, которые позволяли установить значения для предопределенных атрибутов вершин, таких как: геометрические координаты, цвет каждой вершины или её вектор нормали.

С введением шейдеров было принято решение отказаться от предопределенных атрибутов, теперь программист сам может решать, какие атрибуты ему необходимы и как они называются. Поэтому для работы с атрибутами OpenGL использует не их имена, а индексы, которые начинаются с нуля. Для установки всех атрибутов используются общие функции, первым параметром которых является индекс атрибута.

Для установки атрибутов вершин, прежде всего, необходимо указать OpenGL, что для вывода модели будет использоваться атрибут с определенным индексом. Выполняется это с помощью функции glEnableVertexAttribArray:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glEnableVertexAttribArray.xhtml

Далее необходимо описать формат хранения атрибута. Делается это с помощью функции glVertexAttribPointer. Следует отметить, что данный атрибут будет извлекаться из буфера вершин, который был привязан на момент вызова этой функции. Параметры функции во многом совпадают с параметрами функций типа glVertexPointer и могут быть изучены по следующей ссылке:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glVertexAttribPointer.xhtml

Данную последовательность из двух вышеуказанных функций OpenGL (glEnableVertexAttribArray и glVertexAttribPointer) следует применять для всех атрибутов, которые должны быть переданны конвейеру рендеринга для построения полигонов. В частности, хотя в данной лабораторной работе используется только один атрибут – геометрические координаты, рекомендуется передавать в OpenGL все загруженные атрибуты, чтобы не возвращаться к этому вопросу в последующих лабораторных работах.

Следует еще раз ометить, что вся информация о привязанных буферах вершин и буферах индексов, а так же о том, атрибуты с какими индексами используются и как они хранятся, запоминается в ранее созданном и привязанном VAO-объекте. Данная информация указывается один раз, на этом этапе, и будет в дальнейшем использоваться для вывода полигонов на экран в каждом кадре. В этом и заключается назначение VAO-объекта.

**5. «Отвязка» VAO-объекта.**

В завершении рекомендуется «отвязать» VAO-объект, то есть сделать так, чтобы он больше не был текущим, указав в качестве текущего объект с нулевым индексом. Данный этап рекомендуется выполнять, поскольку, как было указанно ранее, такие команды, как glBindBuffer, glEnableVertexAttribArray или glVertexAttribPointer влияют на состояние текущего VAO-объекта, поэтому необходимо позаботится о том, чтобы никакой другой код, например, при загрузке следующего меша, не мог случайно изменить VAO-объект, относящийся к текущему мешу.

## Вывод меша на экран (draw call).

При использовании VAO-объекта этап вывода меша выглядит достаточно просто, поскольку вся информация, которая определяет откуда и как извлекать данные для отрисовки, была ранее запомнена в соответствующем VAO-объекте. В частности, вывод меша осуществялется в соответствии со следующей последовательностью команд:

**1. Привыязка VAO-объекта.**

Прежде всего, необходимо привязать ранее созданный и инициализированный VAO-объект. Выполняется это с помощью ранее рассмотренной функции glBindVertexArray. VAO-объект содержит в себе всю информацию, необходимую для вывода меша на экран, в частности: какие буферы VBO используются в качестве буферов вершин и буферов индексов, а так же то, какие атрибуты вершин используются, как они хранятся и в какие переменные вершинного шейдера передаются.

**2. Вызов команды отрисовки полигонов.**

Далее необходимо вывести полигоны с помощью одной из команд типа glDrawArrays, glDrawElements и им подобных, в зависимости от того, используются ли массивы индексов или нет. Эти команды принято называть draw call, их количество в одном кадре оказывает существенное влияние на производительность приложения.

В данной лабораторной работе используется массив индексов, поэтому необходимо обратиться к функции glDrawElements. Данная функция уже использовалась ранее в проекте прошло семестра, поэтому её параметры должны быть знакомы. Тем не менее, дополнительную информацию можно получить по следующеё ссылке:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glDrawElements.xhtml

Следует отметить, что именно эта команда непосредственно запускает вывод полигонов на экран. При этом атрибуты вершин будут извлекаться из памяти и передаваться в конвейер рендеринга в соответствии с параметрами, определенными в активном VAO-объекте.

**3. «Отвязка» VAO-объекта.**

Завершающим шагом является «отвязка» VAO-объекта, то есть выбор в качестве текущего объекта VAO-объекта с индексом ноль. Выполняется это по тем же самым причинам, что и при загрузке меша, а именно, чтобы последующие команды случайно не испортили информацию в VAO-объекте, например, случайно не привязали другой буфер VBO в качестве буфера верши или буфера индексов.

## Класс для работы с мешем.

Как и для любых других, относительно независимых сущностей, для работы с мешем необходимо организовать отдельный класс, структура которого представлена ниже. При необходимости разрешается вносить изменения, в том числе, добавлять дополнительные интерфейсные и вспомогательные методы, если в этом есть необходимость.

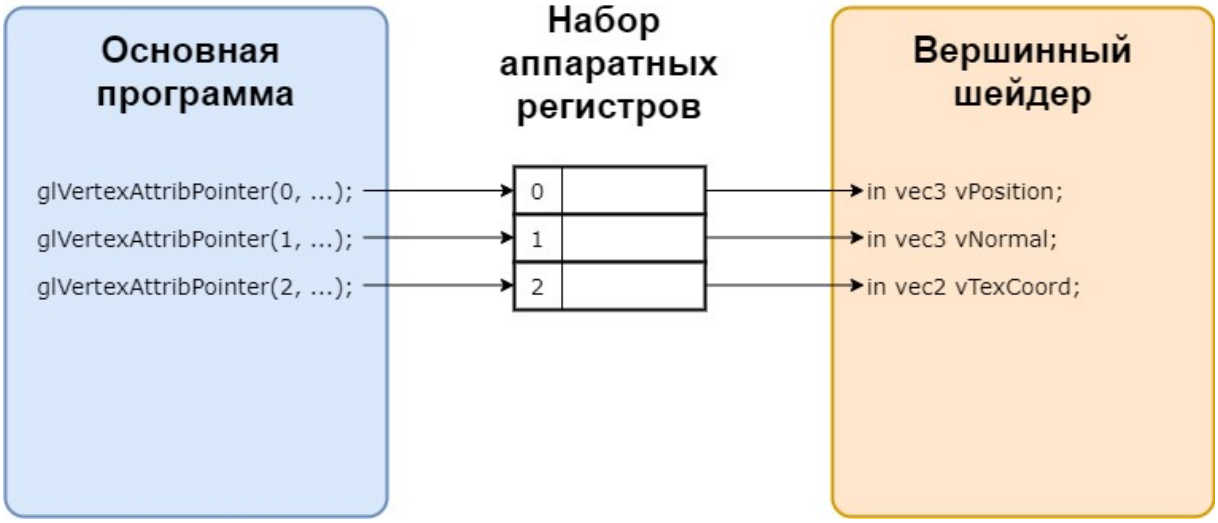
```
// КЛАСС ДЛЯ РАБОТЫ С МЕШЕМ
class Mesh
{
public:
	// конструктор
	Mesh();
	// загрузка меша из внешнего obj-файла
	bool load(std::string filename);
	// вывод меша
	void drawOne();

private:
	// индекс VAO-объекта
	GLuint vao;
	// индекс VBO-буфера вершин
	GLuint vertexBuffer;
	// индекс VBO-буфера индексов
	GLuint indexBuffer;
	// количество вершин в меше
	int vertexCount;

};
```

# Передача атрибутов вершин из основной программы в вершинный шейдер.

Атрибуты вершин из основной программы в вершинный шейдер передаются через набор аппаратных регистров. Каждая переменная вершинного шейдера, объявленная с модификатором in, связанна с регистром, имеющим определенный номер. Данный номер называется индексом (location) соответствующей переменной. Первый параметр функции glVertexAttribPointer указывает, в какую переменную шейдера (а если быть более точным, в какой аппаратный регистр) будет записываться атрибут, определяемый этой функцией. Ниже представлена схема, иллюстрирующая данную взаимосвязь:




Существует несколько способов определить индекс переменной в вершинном шейдере. Первый способ заключается в том, чтобы позволить видеокарте самостоятельно определить связь между аппаратными регистрами и переменными шейдера. После этого, используя функцию glGetAttribLocation, можно получить индекс заданной переменной в конкретном шейдере. Данный индекс можно использовать в функции glVertexAttribPointer для указания того, какие атрибуты вершины будут передаваться в соответствующую переменную вершинного шейдера.

Данный способ, однако, имеет определенные недостатки. Один из недостатков заключается в том, что на этапе создания меша (заполнения VAO-объекта), будет необходимо знать, какой шейдер будет использоваться для вывода этого меша для того, чтобы передавать определенный атрибут в соответствующую переменную шейдера. То есть одним из параметров функции создания меша должен быть шейдер, который будет использоваться для его вывода.

Другим, более существенным недостатком является то, что, если при определенных условиях необходимо будет выводить модель с использованием другого шейдера, потребуется переопределить связи заданные в VAO-объекте функцией glVertexAttribPointer, поскольку в новом шейдере те же самые атрибуты могут быть связанны с другими регистрами.

Для устранения указанных недостатков используется другой способ определения индекса переменной - жесткая привязка атрибут-переменной вершинного шейдера к определенному аппаратному регистру с помощью спецификатора layout. Например, при объявлении переменных так, как показано ниже, будет гарантироваться, что атрибут vPosition всегда будет иметь индекс 0, а vNormal индекс 1, а vTexCoord – индекс 2:

```
layout(location = 0) in vec3 vPosition;
layout(location = 1) in vec3 vNormal;
layout(location = 2) in vec2 vTexCoord;
```

Таким образом, при загрузке модели можно не привязываться к какому-либо конкретному шейдеру, а всегда передавать геометрические координаты в переменную с индексом 0, вектор нормали в переменную с индексом 1, а текстурные координаты в переменную с индексом 2. В этом случае, однако, необходимо гарантировать, что все шейдеры, которые будут использоваться для вывода данного меша, всегда будут размещать переменные соответствующим образом. Данный способ является предпочтительным и должен быть использован как в этой, так и во всех последующих лабораторных работах.

В результате полный текст вершинного шейдера, который используется в третьей лабораторной работе, будет выглядеть следующим образом:

```
1  #version 330 core
2
3  uniform mat4 projectionMatrix;
4  uniform mat4 modelViewMatrix;
5  uniform vec4 color;
6
7  layout(location = 0) in vec3 vPosition;
8  layout(location = 1) in vec3 vNormal;
9  layout(location = 2) in vec2 vTexCoord;
10
11 void main ()
12 {
13     gl_Position = projectionMatrix * modelViewMatrix * vec4 (vPosition, 1);
14 }
```

Текст фрагментного шейдера, на данном этапе, просто устанавливает цвет каждого фрагмента в соответствии со значением uniform-переменной color и выглядит следующим образом:

```
1  #version 330 core
2
3  uniform vec4 color;
4
5  out vec4 outputColor;
6
7  void main (void)
8  {
9      outputColor = color;
10 }
```

# Реализация менеджера ресурсов.

Используя класс Mesh, представленный ранее, можно легко загрузить и вывести на экран произвольное количество объектов. Для хранения мешей можно использовать массив объектов типа Mesh, однако такой подход обладает рядом существенных недостатков, среди которых:

- Существует вероятность того, что один и тот же меш (один и тот же obj-файл) будет загружен несколько раз, что является крайне нерациональным использованием памяти компьютера.
- Поскольку объекты хранятся в разных местах (в том числе и в локальных переменных), существует вероятность того, что некоторые из них не будут должным образом удалены, что, в конечном счете, скажется на производительности или работоспособности программы.

Для того чтобы организовать централизованное хранение, создание и удаление каких-либо объектов используют понятие менеджера. Менеджер, в данном случае, это класс, который берет на себя задачи по управлению другими объектами. В рамках данной лабораторной работы требуется организовать класс ResourceManager, который будет отвечать за централизованное создание, хранение и удаление ресурсов. К ресурсам в этой лабораторной работе относятся только меши, загружаемые из внешних файлов (то есть объекты типа Mesh). Позднее к ресурсам также будут относиться материалы, текстуры и, возможно, некоторые другие сущности.

Менеджер ресурсов часто организуется, используя подход, представленный в библиотеке OpenGL, DevIL, OpenAL и многих других. В частности, все объекты хранятся внутри менеджера, и доступ к ним осуществляется по индексам, предоставляемым самим менеджером. Ниже представлен первый прототип менеджера ресурсов (полная структура класса будет представлена позже):

```
// КЛАСС ДЛЯ РАБОТЫ С МЕНЕДЖЕРОМ РЕСУРСОВ
class ResourceManager
{
public:
    // Загрузка одного меша из указанного obj-файла
    // Возвращаемое значение - индекс меша в менеджере ресурсов
    int loadMesh(std::string filename);

    // Получение меша по его индексу
    // Если такого меша нет (недействительный индекс) возвращается nullptr
    Mesh* getMesh(int index);

private:
    // вектор для хранения всех мешей
    std::vector<Mesh> meshes;

};
```

Для хранения всех мешей менеджер ресурсов имеет приватное поле – вектор, хранящий все меши. Соответственно, каждый меш имеет свой собственный индекс в данном контейнере. Поскольку поле объявлено приватным, доступ к нему имеет только сам менеджер ресурсов. Это гарантирует обработку мешей (их создание, удаление и так далее) только данным классом.

Менеджер ресурсов обладает двумя основными методам:

1. **loadMesh** – принимает в качестве параметра имя obj-файла, который необходимо загрузить. Данный метод создает один меш, загружает в него указанный файл и помещает меш в вектор. После этого функция возвращает индекс меша в данном контейнере. Используя полученный индекс, в дальнейшем можно получить указатель на требуемый меш.
2. **getMesh** – возвращает указатель на меш по его индексу. Полученный указатель можно использовать для вывода меша на экран. При этом сам меш по-прежнему хранится в векторе, объявленном внутри менеджера ресурсов.

Благодаря централизованному хранению мешей менеджер ресурсов может обеспечить дополнительную функциональность (не все из указанного ниже требуется реализовывать в рамках данного проекта):

1. Предотвращение загрузки одинаковых мешей. В случае если пользователь запрашивает загрузку меша, который был ранее загружен и уже содержится в контейнере, вместо загрузки еще одной копии возвращается индекс ранее загруженного меша. Для удобного определения того, был ли ранее загружен меш, можно воспользоваться ассоциативным контейнером std::map также, как это было сделано с uniform-переменными в одной из предыдущих лабораторных работ. В рамках лабораторной работы необходимо реализовать описанную функциональность.

2. Менеджер ресурсов может содержать метод для централизованного удаления всех мешей. Метод может быть вызван отдельно или вызываться в деструкторе менеджера ресурсов.

3. Отложенная (асинхронная) загрузка ресурсов. Метод загрузки ресурсов может запомнить имя файла, который требуется загрузить и сразу вернуть индекс, выделенный для данного ресурса. При этом сама загрузка будет осуществляться асинхронно в отдельном потоке выполнения. Это позволит получить выигрыш в производительности в многоядерных системах.

4. Подсчет количества ссылок на ресурс. При динамическом создании и удалении графических объектов, каждый из которых использует некоторый меш, менеджер ресурсов может подсчитывать количество ссылок на меш. То есть каждый раз, когда вызывается метод loadMesh, количество ссылок на соответствующий меш увеличивается на единицу. Каждый раз, когда объект удаляется (например, вызывается метод deleteMesh, который здесь не представлен) количество ссылок уменьшается. Непосредственное удаление меша и освобождение памяти происходит только в том случае, если количество ссылок уменьшилось до нуля.

Как было сказано выше, наличие единого объекта, занимающегося созданием, хранением и удалением мешей имеет целый ряд преимуществ и позволяет существенно упростить логику программы. При этом возникает вопрос, как гарантировать то, что менеджер ресурсов действительно будет существовать в единственном экземпляре? Что помешает программисту создать несколько копий менеджера ресурсов, каждый из которых будет хранить свой вектор мешей?

Одним из вариантов решения данной проблемы являются административные меры. При разработке программы необходимо указать, что требуется создать только один объект данного класса, определить его имя и донести эту информацию до всех разработчиков, участвующих в проекте. Данный подход не дает, однако, стопроцентной гарантии. Кроме того, всем разработчиком требуется знать имя этого уникального объекта. Объект должен быть глобальным и доступным из любого модуля или класса, который собирается обращаться к менеджеру ресурсов.

Другим вариантом, который необходимо реализовать в данной лабораторной работе, является использование шаблона проектирования singleton (одиночка). Одиночка — это порождающий шаблон проектирования, гарантирующий, что в приложении будет единственный экземпляр некоторого класса, и предоставляющий глобальную точку доступа к этому экземпляру. В C++ наиболее часто используется синглтон Майерса. Ниже приведен пример реализации класса, взятый с сайта Википедии:

```
// КЛАСС, РЕАЛИЗУЮЩИЙ ШАБЛОН СИНГЛТОН МАЙЕРСА
class OnlyOne
{
public:
    static OnlyOne& Instance()
    {
        static OnlyOne theSingleInstance;
        return theSingleInstance;
    }
private:
    OnlyOne() {}
    OnlyOne(const OnlyOne& root) = delete;
    OnlyOne& operator=(const OnlyOne&) = delete;
};
```

Прежде всего, следует обратить внимание на то, что конструктор по умолчанию объявлен приватным и нет ни одного публичного конструктора. В результате невозможно создать объект данного класса вне самого класса. Это гарантирует, что ни один программист, случайно или специально, не сможет создать объект данного класса. Как же тогда использовать данный класс?

Для этого служит статичный метод Instance, который возвращает ссылку на объект данного класса. Сам объект является статичным и объявлен внутри метода Instance, то есть он будет создан один раз в момент первого обращения к данному методу. Все операции с данным классом производятся с помощью обращения к методу Instance, то есть все модули будут иметь дело с одним и тем же объектом, объявленным как статичная переменная внутри этого статичного метода.

Комбинируя вышеприведенную функциональность, которую должен реализовать менеджер ресурсов, и шаблон проектирования singleton, получаем класс для работы с ресурсами, который выглядит так, как показано ниже. В лабораторной работе необходимо организовать именно эту версию класса:

```
// КЛАСС ДЛЯ РАБОТЫ С МЕНЕДЖЕРОМ РЕСУРСОВ (РЕАЛИЗОВАН НА ОСНОВЕ ШАБЛОНА SINGLTON)
class ResourceManager
{
public:
	// Статик-метод для получения экземпляра менеджера ресурса.
	// Всегда будет возвращена ссылка на объект, хранящийся в единственном экземпляре.
	static ResourceManager& instance()
	{
		static ResourceManager ResourceManager;
		return ResourceManager;
	}

	// Загрузка одного меша из указанного obj-файла
	// Возвращаемое значение - индекс меша в менеджере ресурсов
	int loadMesh(std::string filename);

	// Получение меша по его индексу
	// Если такого меша нет (недействительный индекс) возвращается nullptr
	Mesh* getMesh(int index);

private:
	// конструктор по умолчанию (объявлен приватным)
	// в результате нельзя создать ни одного объекта данного класса вне самого класса
	ResourceManager() {};
	// конструктора копирования нет (удален)
	ResourceManager(const ResourceManager& v) = delete;
	// оператора присваивания нет (удален)
	ResourceManager& operator=(const ResourceManager& v) = delete;

private:
	// вектор для хранения всех мешей
	std::vector<Mesh> meshes;
	// map для хранения соответствия между именем запрашиваемого файла
	// и индексом в контейнере meshes
	std::map <std::string, int> meshes_id;

};
```

# Модификация графического объекта.

В рамках четвертой лабораторной работы требуется организовать вывод нескольких графических объектов на экран, при этом графические объекты могут быть представлены как разными мешами, так и использовать один и тот же меш.

Ранее для представления графического объекта был реализован класс GraphicObject, который хранил цвет объекта и его матрицу модели. На данном этапе к классу также добавляется информация об используемом меше. При этом класс хранит не сам меш, а его индекс в менеджере ресурсов. Иными словами, класс для оперирования графическими объектами должен выглядеть следующим образом:

```
// КЛАСС ДЛЯ РАБОТЫ С ГРАФИЧЕСКИМ ОБЪЕКТОМ
class GraphicObject
{
public:
	// конструктор по умолчанию
	GraphicObject();

	// установить цвет объекта
	void setColor(glm::vec4 &color);
	// установить позицию объекта
	void setPosition(glm::vec3 &position);
	// установить угол поворота в градусах относительно оси OY по часовой стрелке
	void setAngle(float degree);
	// установить идентификатор используемого меша
	void setMeshId(int id);

	// получить различные параметры
	glm::vec4& getColor();
	glm::mat4& getModelMatrix();
	int getMeshId();

private:
	// идентификатор используемого меша
	int meshId;
	// цвет объекта
	glm::vec4 color;
	// матрица модели (задает позицию и ориентацию)
	glm::mat4 modelMatrix;

};
```

Следует обратить внимание на то, что класс не имеет метода для собственной загрузки (например, на основании параметров, заданных во внешнем файле) или метода для вывода на экран, поскольку оба этих метода являются достаточно сложными и не относятся к его функционалу. В частности, в следующих лабораторных работах будут реализованы отдельные классы, ответственные за инициализацию графических объектов и за их вывод на экран. Данный класс является скорее контейнером, объединяющим все необходимые данные, для представления графического объекта, в одном месте.

Для лучшего понимая предназначения всех классов и способа их использования, рассмотрим пример инициализации и оперирования графическими объектами для получения изображения на экране. Для начала укажем основные глобальные данные, которые используются в примере к лабораторной работе:

```
// ИСПОЛЬЗУЕМАЯ КАМЕРЫ
Camera camera;
// ИСПОЛЬЗУЕМЫЙ ШЕЙДЕР (ПОКА ТОЛЬКО ОДИН)
Shader shader;
// СПИСОК ГРАФИЧЕСКИХ ОБЪЕКТОВ ДЛЯ ВЫВОДА НА ЭКРАН
vector<GraphicObject> graphicObjects;
```

Перед началом использования (то есть перед выводом объектов на экран) графические объекты необходимо инициализировать. Для этого удобнее написать отдельную функцию, например так, как показано ниже:

```
// вспомогательная  функция для инициализации графических объектов
void initGraphicObjects()
{
    // ссылка на менеджер ресурсов (для удобства)
    ResourceManager &rm = ResourceManager::instance();
    // временная переменная для хранения идентификаторов меша
    int meshId = -1;
    // временная переменная для представления графического объекта
    GraphicObject graphicObject;

    // добавление графического объекта
    meshId = rm.loadMesh("MESHES\\buildings\\house_2.obj");
    graphicObject.setMeshId(meshId);
    graphicObject.setColor(vec4(0.2, 0.2, 0.2, 1));
    graphicObject.setPosition(vec3(0, 0, 0));
    graphicObject.setAngle(0.0);
    graphicObjects.push_back(graphicObject);

    // добавление графического объекта
    meshId = rm.loadMesh("MESHES\\natures\\big_tree.obj");
    graphicObject.setMeshId(meshId);
    graphicObject.setColor(vec4(0.2, 0.8, 0.2, 1));
    graphicObject.setPosition(vec3(7.5, -0.75, 2.5));
    graphicObject.setAngle(0.0);
    graphicObjects.push_back(graphicObject);

    // добавление графического объекта
    meshId = rm.loadMesh("MESHES\\natures\\big_tree.obj");
    graphicObject.setMeshId(meshId);
    graphicObject.setColor(vec4(0.2, 0.8, 0.2, 1));
    graphicObject.setPosition(vec3(-7.5, -0.75, 2.5));
    graphicObject.setAngle(0.0);
    graphicObjects.push_back(graphicObject);

    // добавление графического объекта
    meshId = rm.loadMesh("MESHES\\vehicles\\police_car.obj");
    graphicObject.setMeshId(meshId);
    graphicObject.setColor(vec4(0.2, 0.2, 1.0, 1));
    graphicObject.setPosition(vec3(+4.5, -2.15, +6.5));
    graphicObject.setAngle(-115.0);
    graphicObjects.push_back(graphicObject);

    // добавление графического объекта
    meshId = rm.loadMesh("MESHES\\vehicles\\police_car.obj");
    graphicObject.setMeshId(meshId);
    graphicObject.setColor(vec4(0.23, 0.23, 1.0, 1));
    graphicObject.setPosition(vec3(+4.25, -2.15, +10.5));
    graphicObject.setAngle(+105.0);
    graphicObjects.push_back(graphicObject);

    // добавление графического объекта
    meshId = rm.loadMesh("MESHES\\vehicles\\jeep.obj");
    graphicObject.setMeshId(meshId);
    graphicObject.setColor(vec4(0.95, 0.13, 0.13, 1));
    graphicObject.setPosition(vec3(-1.25, -2.15, +9.0));
    graphicObject.setAngle(+170.0);
    graphicObjects.push_back(graphicObject);

}
```

Далее приводится полный текст функции display, которая обращается к ранее инициализированным данным для построения изображения на экране:

```
// функция вызывается при перерисовке окна
// в том числе принудительно, по команде glutPostRedisplay
void display()
{
	// очистка буфера кадра
	glClearColor (1.0, 1.0, 1.0, 1.0);
	glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );

	// включение теста глубины (на всякий случай)
	glEnable(GL_DEPTH_TEST);

	// вывод полигонов в виде линий с отсечением нелицевых граней
	glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
	glEnable(GL_CULL_FACE);
	glCullFace(GL_BACK);

	// активируем шейдер, используемый для вывода объекта
	shader.activate();

	// устанавливаем матрицу проекции
	mat4& projectionMatrix = camera.getProjectionMatrix();
	shader.setUniform("projectionMatrix", projectionMatrix);

	// получаем матрицу камеры
	mat4& viewMatrix = camera.getViewMatrix();

	// выводим все объекты
	for (auto& graphicObject : graphicObjects) {

		// устанавливаем матрицу наблюдения модели
		mat4 modelViewMatrix = viewMatrix * graphicObject.getModelMatrix();
		shader.setUniform("modelViewMatrix", modelViewMatrix);

		// устанавливаем цвет
		shader.setUniform("color", graphicObject.getColor());

		// выводим меш
		int meshId = graphicObject.getMeshId();
		Mesh *mesh = ResourceManager::instance().getMesh(meshId);
		if (mesh != nullptr) mesh->drawOne();
	}

	// меняем передний и задний буферы цвета
	glutSwapBuffers();

	// вычисление FPS и его вывод в заголовок окна
	char windowTitle[128];
	int FPS = getFps();
	sprintf_s(windowTitle, 128, "Laba_04 [%i FPS]", FPS);
	glutSetWindowTitle(windowTitle);

}
```

# Задание к лабораторной работе.

Лабораторная работе №4 строится на основе предыдущей работы. К программе предъявляются следующие требования:

1. Необходимо реализовать класс Mesh для загрузки и вывода меша на экран. Класс должен иметь структуру, приведенную выше. Загрузка должна осуществляться из obj-файлов, а вывод выполняться с использованием буфера вершин, буфера индексов и VAO-объекта.
2. Необходимо реализовать класс ResourceManager для централизованного управления ресурсами (мешами). Класс должен иметь структуру, приведенную выше, и должен быть реализован на основе шаблона проектирования singleton. При загрузке меша необходимо проверять, был ли данный меш загружен ранее. В случае если меш уже загружен, необходимо вернуть идентификатор ранее загруженного меша, новый меш, в этом случае, создаваться не должен.
3. Необходимо модифицировать класс GraphicObject, добавив в него идентификатор используемого меша. Класс должен иметь структуру, описанную выше.
4. В завершении требуется реализовать вывод нескольких (не менее 5) объектов, часть из которых повторяется. Вывести на консоль отладочную информацию, показывающую, что класс ResourceManager справляется со своей задачей, в частности, следит за тем, чтобы повторно не загружать уже загруженные меши.
5. Ресурсы для демонстрации программы (меши и шейдеры) необходимо взять из демонстрационного примера.

Отчет к лабораторной работе должен быть сформирован на основе шаблона, представленного на следующей странице. При оформлении отчета необходимо внести свою фамилию, имя, отчество, номер группы, а также заполнить таблицу. При заполнении таблицы необходимо дать краткое описание реализованным модулям, указать количество строк каждого модуля, а также подсчитать общее количество строк во всем проекте.

# Архитектура графических систем (6 семестр)

**Выполнил: Иванов Иван Иванович**

Группа: 00-00

Проверил: Галибин Сергей Владимирович

## Задание:

В лабораторной работе №4 необходимо организовать вывод на экран нескольких графических объектов, каждый из которых представлен некоторым мешем, загруженным из внешнего obj-файла. Для вывода меша на экран необходимо использованием буферы OpenGL и VAO-объекты. Для эффективного управления мешами необходимо реализовать менеджер ресурсов.

## Защита лабораторной работы:

Для защиты лабораторной работы необходимо знать:

1. **[КГ]** Что такое меш, полигон, массив вершин и массив индексов.
2. **[КГ]** Что такое атрибуты вершины, какие атрибуты используются в данной лабораторной работе.
3. **[КГ]** Что такое буферы OpenGL и VAO-объекты, какую роль они играют при выводе меша на экран.
4. **[КГ]** Какие два типа буферов используются для вывода полигонов.
5. **[КГ]** Как происходит процесс передачи атрибутов вершин в вершинный шейдер.
6. **[CPP]** Что такое статические функции класса.
7. **[CPP]** Как реализуется шаблон проектирования Singlton в C++.
8. **[CPP]** Для чего необходим менеджер ресурсов.

## Состав проекта:

| **Название модуля и его назначение** | **Количество строк** | |
|---|---|---|
| | ***.h** | ***.cpp** |
| main - основная программа | | |
| Shader - | | |
| Camera - | | |
| GraphicObject - | | |
| Mesh - | | |
| ResourceManager - | | |
| | | |